

Evolution API · Robust-0414

Production-hardening fork of EvolutionAPI/evolution-api · version 2.3.8-robustness.0414.5 · 42 commits across 30+ files · `multi-arch: linux/amd64, linux/arm64`

 **Docker Hub** · `shimonuziel/evolution-api`

TAG	USE
robust-0414.5	Production — pinned, immutable
0414.5	Same image, short alias
v2.3.8-robustness.0414.5	Same image, matches git tag
robust	Rolling — dev only, NOT prod compose

digest:
sha256:a08125ffe95f39ffd77c757a644474b046f845589785700ffead6cc7a87f328a

 **Docs (read in order)**

FILE	AUDIENCE
SUMMARY.md	Master index — start here
DEPLOY.md	Operator runbook (compose, env, verify)
ROBUSTNESS.md	23 env vars + tuning playbook
CHANGELOG.md	Per-release log (5 releases)
.env.example	Updated template w/ new vars

What changed (compared to `evoapicloud/evolution-api:latest`)

Lifecycle & HTTP

- Graceful SIGTERM / SIGINT shutdown (25s budget, closes HTTP → Baileys → Prisma → Redis → Chatwoot pg)
- HTTP server timeouts (keepAlive, headers, request, maxConnections)
- helmet + express-rate-limit (600/min/IP, tunable)
- Conditional trust proxy (anti-spoof)
- Bootstrap failures exit cleanly (no Prisma stack dump)
- uncaughtException → Sentry + exit(1)
- New endpoints: `/health/live`, `/health/ready`, `/metrics`

Baileys / WhatsApp

- Removed double-reconnect race (zombie instance bug)
- Per-event try/catch in event handler
- Pre-send connection guard + bounded retry on transport errors
- saveCreds() per-instance mutex
- Per-instance groupMetadataCache scoping
- Status broadcast throttling
- Absolute QR pairing budget (5 min)
- Media inlining size cap (50 MB)
- msgRetryCounterCache TTL (was unbounded)
- Cron jobs tracked + stopped on logout
- Zombie-instance detector with reload race guard

Integrations & ops

- 30s timeouts on every chatbot HTTP call (OpenAI, Dify, Flowise, N8N, Typebot, EvoAI, EvolutionBot)
- Per-endpoint circuit breakers (opossum)
- OpenAI Assistant runs bounded by wall-clock
- WA Business post() throws on failure (was silent)
- Chatwoot pg pool disposed on shutdown
- Event manager runs all 7 transports concurrently
- Webhook retries fire-and-forget
- WebSocket connection caps (global + per-IP)
- Prisma pool, connect, statement timeouts
- Redis reconnect strategy + log throttle
- Prometheus `/metrics` endpoint
- npm audit: **66** → **5** findings (dev-only)

Behavior changes worth knowing

- Body limit 50 MB** (was 136 MB) — tunable via `REQUEST_BODY_LIMIT_MB`
- Sends to disconnected instances throw 400** (used to silently queue) — opt out with `ENSURE_CONNECTED_ON_SEND=false`
- WA Business sends throw on failure** (used to return undefined)
- SIGTERM = graceful shutdown** (~25s) — set `stop_grace_period: 30s` in compose
- uncaughtException exits the process** by default — disable with `EXIT_ON_UNCAUGHT=false`

Smoke-test before merge

- Compose stack up with real Postgres + Redis
- `/health/live 200 · /health/ready 200 · /metrics 200`
- Pair a WhatsApp instance via QR; send / receive text + media
- Force Redis kill mid-flight → logs throttle, no crash
- Force Postgres kill mid-flight → `/health/ready` → 503
- `kill -TERM $pid` → graceful shutdown sequence within 25 s
- Open 51 WS connections from 1 IP → 51st rejected

 Honest gaps

- Zero unit tests — all guards are reasoned-about-correctness only
- Never paired a real WA instance during this build
- No load test; no live SIGTERM test against connected instances
- Custom Prometheus counters declared but increment hooks not yet wired
- Session-store concurrency outside saveCreds mutex not audited

 Quickstart

STEP	COMMAND
1. Pull image	<code>docker pull shimonuziel/evolution-api:robust-0414.5</code>
2. Edit compose	<code>image: shimonuziel/evolution-api:robust-0414.5</code> <code>stop_grace_period: 30s</code>
3. Bring up	<code>docker compose up -d</code>
4. Verify	<code>curl -sf localhost:8080/health/live</code>

Full step-by-step in `DEPLOY.md`. Don't use `:robust` or `:latest` in production compose — those tags float.